HASH STRING — SUFFIX ARRAY

- Alexandra Melgar Loza A01770569
- Ileana Tapia Castillo A01773374
- María Fernanda Valdovinos Mañón A01773977
- Natalia Sosa Torres A01770703
- Tabata Carolina Salinas Valdez A01770901
- Yael Sebastián González Hernández A01770992
- Alessandro Daturi Miranda A01773901
- Alan Joseph Salazar Segura A01773491

PRECÓMPUTO HASH

O(n)

CONSTRUCCIÓN SUFFIX ARRAY

O(n log n)

Comparar y ordenar subcadenas sin leerlas letra por letra.

Comparar dos subcadenas de longitud L, carácter por carácter, cuesta O(L). Repetido miles de veces —búsqueda de patrones, deduplicación, ordenar sufijos, contar subcadenas distintas— ese costo se arrastra en cada paso. Dos técnicas lo resuelven desde ángulos distintos: el **hashing** mapea cada subcadena a un entero y compara por igualdad; el **suffix array** ordena todos los sufijos y compara por orden lexicográfico real.

ANTES / DESPUÉS

- Fuerza bruta — $O(n^2 \cdot L)$

  Comparar todos los pares de subcadenas o sufijos exige recorrerlos cada vez.

- Con hashing — $O(1)$

  Un precómputo O(n) permite comparar cualquier par de subcadenas en tiempo constante.

- Con suffix array — $O(\log n)$

  Una construcción O(n log n) permite ordenar, buscar patrones y calcular LCP eficientemente.

FUNCIÓN HASH POLINOMIAL

Rolling hash.

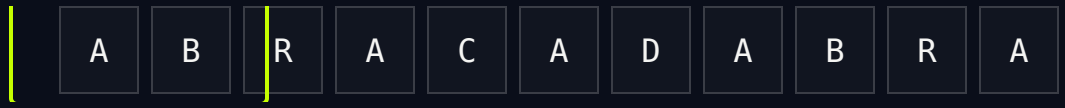
$$H(s) = (s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_{n-1} \cdot p^{n-1}) \bmod m$$

p es un primo cercano al alfabeto (31, 53); m un módulo grande (1e9+7). El hash de s[l..r] sale por resta de prefijos en O(1):

$$\text{hash}(l, r) = (H[r] - H[l-1] \cdot p^{r-l}) \bmod m$$

VENTANA DESLIZANTE SOBRE EL TEXTO

El hash de cada ventana de longitud fija se deriva del anterior sin recalcular desde cero — de ahí "rolling".



la ventana verde recorre "ABRACADABRA" comparando cada hash contra el patrón

Tiempo, espacio y riesgo de error.

Operación	Tiempo	Espacio
Precómputo de prefijos	O(n)	O(n)
Hash de una subcadena	O(1)	O(1)
Comparación de dos subcadenas	O(1)	O(1)
Colisión falsa (un módulo)	P ≈ 1/m	—
Colisión falsa (doble módulo)	P ≈ 1/m²	—

POR QUÉ DOBLE MÓDULO

El hashing es O(1) en tiempo siempre — lo que cambia con la técnica no es la velocidad, sino la probabilidad de que dos subcadenas distintas produzcan el mismo hash. Con un solo (p, m) esa probabilidad es ~1/m; con dos pares independientes cae a ~1/m² — la diferencia entre una implementación de examen y una robusta frente a un juez adversarial.

- Rigor formal →

  Una colisión no cuesta tiempo extra, cuesta corrección: el algoritmo sigue siendo O(1), pero puede dar un falso positivo si el módulo es fijo y conocido por un adversario.

Fortalezas y limitaciones.

CUÁNDO ES ÓPTIMO

Velocidad +

Comparación de subcadenas en O(1) tras un único precómputo O(n).

Simplicidad +

Mucho más simple de implementar y depurar que Suffix Array o Suffix Automaton.

Memoria +

Solo dos arreglos de tamaño n — bajo overhead frente a estructuras de sufijos.

DESVENTAJAS FRENTE A OTRAS ALTERNATIVAS

Probabilístico –

Existe riesgo, aunque pequeño, de colisión falsa; nunca es 100% determinístico.

Sin orden –

No ordena subcadenas lexicográficamente — ahí Suffix Array es la alternativa correcta.

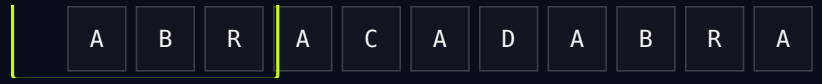
Hackeable –

Con módulo fijo y conocido, un adversario puede construir colisiones deliberadas.

CASO 01

Búsqueda de patrones — Rabin–Karp

Se hashea el patrón "ABRA" y cada ventana de "ABRACADABRA"; comparar dos hashes es O(1), así que recorrer todo el texto cuesta O(n).




```
posición 0 → hash coincide ✓
posición 3 → hash distinto
posición 7 → hash coincide ✓
```

CASO 02

Longest Common / Palindromic Substring

Se busca la longitud L más grande que cumple la propiedad, probando valores de L por búsqueda binaria; cada prueba compara hashes en O(n), total O(n log n).

```
L = 8 → ¿existe coincidencia de esa longitud?
no → probar L = 4
sí → probar L = 6
no cabe más → respuesta: L = 5
```



LECTURA DEL CÓDIGO

Tres pasos, tres complejidades.

- __init__ — O(n)

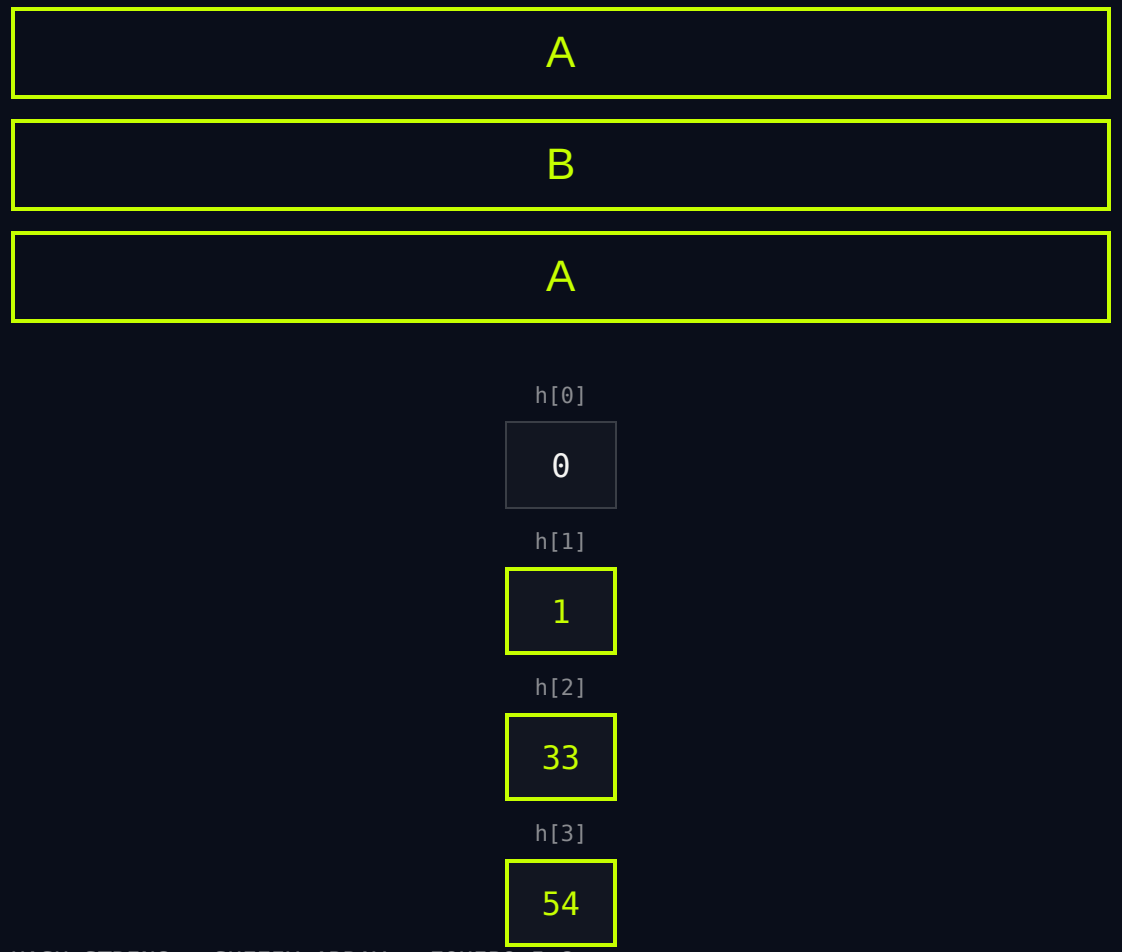
  Se ejecuta una sola vez: construye prefijos y potencias de ambos módulos.

- get_hash — O(1)

  Resta de prefijos — la operación que justifica todo el precómputo previo.

Así se llena y se consulta h[ ].

s = "ABA", p = 31, m = 97. Cada carácter empuja h[i] = (h[i-1]·p + valor) mod m. Pedir hash(1, 3) = hash("BA") solo resta contra el arreglo ya construido, sin releer "BA".



(54 − 1×88) mod 97 = **63** = hash("BA") — un solo paso, sin tocar "B" ni "A" de nuevo.

Casos de prueba.

Cada caso valida una propiedad distinta: búsqueda de patrón, igualdad de subcadenas y no-igualdad cuando el contenido difiere.

```
>>> rabin_karp_search("abracadabra", "abra")
[0, 7] ✓ coincide con las posiciones esperadas
>>> h = StringHasher("abcabc")
>>> h.equal(0, 3, 3, 6) # "abc" vs "abc"
True ✓
>>> h.equal(0, 3, 1, 4) # "abc" vs "bca"
False ✓
```

Todos los sufijos, ordenados.

El suffix array SA de una cadena s de longitud n es la permutación de los índices 0..n−1 tal que los sufijos s[SA[0]:], s[SA[1]:], … quedan ordenados lexicográficamente. En vez de comparar por hash, se compara por orden real de caracteres.

Construcción por duplicación de rango (Manber–Myers): en la iteración k, se ordena por los primeros 2^k caracteres, duplicando k en cada paso hasta cubrir n.

SUFIJOS DE "BANANA"

Los 6 sufijos ordenados lexicográficamente y su posición original:

5	**A**	SA[0]
3	**ANA**	SA[1]
1	**ANANA**	SA[2]
0	**BANANA**	SA[3]
4	**NA**	SA[4]
2	**NANA**	SA[5]

Construcción y consulta.

Operación	Tiempo	Espacio
Construcción ingenua (ordenar sufijos)	$O(n^2 \log n)$	$O(n^2)$
Duplicación de rango — sort() (este código)	$O(n \log^2 n)$	$O(n)$
Duplicación de rango — con radix sort	$O(n \log n)$	$O(n)$
Construcción SA-IS (lineal)	$O(n)$	$O(n)$
LCP array (algoritmo de Kasai)	$O(n)$	$O(n)$
Búsqueda de patrón (binary search)	$O(m \log n)$	$O(1)$ extra

NOTA DE RIGOR: SORT VS RADIX SORT

Nuestro código ordena cada iteración con sort() genérico (comparación): O(n log n) por iteración × O(log n) iteraciones = **$O(n \log^2 n)$**. El Manber–Myers "clásico" en O(n log n) reemplaza ese sort por un radix sort sobre los pares (rank, next-rank), que ordena cada iteración en O(n).

- **Determinismo** →

  A diferencia del hashing, el resultado siempre es exacto: no hay probabilidad de error.

Fortalezas y limitaciones.

CUÁNDO ES ÓPTIMO

Determinístico +

Sin probabilidad de colisión: el resultado es siempre exacto, sin importar el adversario.

Orden real +

Permite comparar subcadenas lexicográficamente, algo que el hashing no resuelve.

Versátil con LCP +

Subcadena repetida más larga, conteo de subcadenas distintas y LCS entre dos strings, todo en O(n) adicional.

DESVENTAJAS FRENTE A OTRAS ALTERNATIVAS

Implementación –

Mucho más código que un rolling hash; la versión lineal (SA-IS) es notoriamente intrincada.

Overhead de memoria –

Requiere varios arreglos auxiliares (rank, tmp, LCP) además del propio SA.

Consulta más costosa –

Buscar un patrón cuesta O(m log n), frente al O(1) del hashing tras precómputo.

CASO 01

Búsqueda de patrones — binary search sobre SA

Como los sufijos están ordenados, buscar si un patrón p aparece en s equivale a hallar su rango en el SA con dos búsquedas binarias, comparando p contra sufijos: costo O(m log n).

```
"ANA" < sufijo → buscar mitad izquierda
"ANA" > sufijo → buscar mitad derecha
rango encontrado → todas las ocurrencias son contiguas en SA
```

CASO 02

LCP array — subcadena repetida más larga

El valor máximo del LCP array es, directamente, la longitud de la subcadena repetida más larga. El conteo de subcadenas distintas sale de n(n+1)/2 menos la suma del LCP — ambas en O(n) tras construir SA y LCP.

SA[1] ANA	LCP=1
SA[2] ANANA	**LCP=3**
SA[3] BANANA	LCP=0
SA[4] NA	LCP=0
SA[5] NANA	LCP=2

max(LCP) = 3 → la subcadena repetida más larga en "BANANA" es "ANA".

LECTURA DEL CÓDIGO

Dos funciones, dos complejidades.

build_suffix_array	$O(n \log^2 n)$
log n iteraciones, cada una ordena con sort() en O(n log n). Con radix sort sobre (rank, next) bajaría a O(n log n) total.	
kasai_lcp	$O(n)$
Explota que h nunca baja más de 1 entre iteraciones consecutivas — amortizado lineal.	

De sufijos crudos a SA.

Con k=1 se ordena por 2 caracteres; con k=2 por 4; con k=4 ya se cubren los 6 caracteres y el orden queda estable.



SA = [5, 3, 1, 0, 4, 2]

Casos de prueba.

Se valida el propio SA, el LCP array derivado y la búsqueda de patrón sobre el arreglo ordenado.

```
>>> sa = build_suffix_array("banana")
[5, 3, 1, 0, 4, 2] ✓
>>> kasai_lcp("banana", sa)
[0, 1, 3, 0, 0, 2] ✓ max = 3 → "ana"
>>> search_pattern("banana", sa, "ana")
[1, 3] ✓ ocurre en las posiciones 1 y 3
```



GRACIAS.